

写给不懂编程的人

AgentLoom 是怎么搭起来的

这份文档假设你不写代码，也没听过 Zod、NestJS、Drizzle 这些名字。它会从「这个系统到底在干什么」讲起，每遇到一个新名词就先用大白话解释一遍，再讲它在系统里起什么作用、为什么选它而不是别的。

读完之后，你应该能向别人讲清楚：这个系统由哪几块组成、它们各自解决什么问题、以及为什么当初这么选。

读前须知

这份文档怎么用

你设计了这个系统，但代码基本是 AI 写的。所以你现在的处境很具体：你知道每一块该干什么，但不知道它们实际上是怎么干的。这份文档专门补这一层。

它跟同目录下另外几份 PDF 不一样。那几份是给已经懂技术的人看的速记卡——一句话一个结论，术语不解释，密度极高。那种写法对你没用，因为它假设读者早就知道 Zod 是什么、RLS 是什么。这份文档反过来：宁可啰嗦，也不跳步。

三种方框的含义

正文里会反复出现三种带边框的段落，它们的作用不同：

名词解释 TERM

蓝色左边框的框，是某个技术名词第一次出现的地方。里面用大白话讲它是什么、干什么用的。看到这种框可以慢一点读，后面的正文会直接用这个词，不再重复解释。

打个比方

灰色边框的框，是类比。整份文档会反复用「一间餐厅」来比这个系统——因为餐厅里有接单、排队、后厨、独立灶台、食材仓库、包间，正好能对上系统里的每一块。类比不精确，但能帮你先建立整体印象。

不这样会出什么事

橙色左边框的框，讲的是「如果不这么做会怎样」。技术选择的理由往往藏在这里——不是因为某个技术好，而是因为不用它会踩到某个具体的坑。这类框是理解「为什么」的关键。

建议的读法

如果你时间紧，只读[第一部分](#)和[第五部分](#)：前者给你整体印象，后者跟着一次真实的「点击运行」把整个系统走一遍。这两部分读完，你已经能跟人聊得下去了。

如果你要准备面试，重点是[第七部分](#)——它讲的不是技术，是怎么诚实地描述「哪部分是你的贡献」。这部分建议完整读，因为它涉及一些反直觉的判断。

中间的第二、三、四部分是词典和逐项拆解，可以在需要时回头查。

一句话预告

如果只让你记住一句话，那就是这句：这个系统的难点不在于「让 AI 干活」，而在于「让很多个 AI 按正确的顺序干活，出错时知道错在哪，而且不会互相干扰、也不会把机器搞坏」。后面所有的技术选择，都是为了这一句话里的某个词。

目录

这份文档讲什么

第一部分 · 先建立整体印象

- 1 这个系统到底在干什么：一个完整的例子
- 2 把它想象成一间餐厅

第二部分 · 名词表

- 3 编程语言与运行环境
- 4 框架与工具库
- 5 存数据的四个地方

第三部分 · 六个组成部分

- 6 画布：你看到的那个界面
- 7 服务器：接单和派活的那一层
- 8 队列：为什么不能「来了就干」
- 9 智能体：跟直接问 AI 差在哪
- 10 沙箱：为什么不敢让 AI 直接在服务器上跑命令
- 11 为什么要四种数据库而不是一种

第四部分 · 为什么用这些技术

- 12 为什么整个项目用 TypeScript
- 13 Zod 是什么，为什么它这么重要
- 14 后端为什么用 NestJS 加 Fastify
- 15 为什么用 Drizzle 碰数据库
- 16 前端那一堆库各自在干什么
- 17 为什么会掺进 Rust 和 Go

第五部分 · 走一遍完整流程

- 18 点一次「运行」，系统里发生了什么

第六部分 · 三处最费心的设计

- 19 怎么保证三个端说的是同一种话
- 20 笼子是怎么焊住的
- 21 怎么保证 A 公司看不到 B 公司的数据

第七部分 · 面试时怎么说

- 22 哪部分算你的贡献
- 23 已知还没修的问题

全文出现的技术名词都在第二部分有解释。如果读到某个词想不起来是什么，回第二部分查。

第 一 部 分

先建立整体印象

不讲任何技术名词，先说清这个系统在干什么、以及为什么它比「直接问 AI」复杂得多。

第 1 章

这个系统到底在干什么

先不谈架构。设想一个具体任务：每周一早上，自动生成一份上周的项目周报，发到团队邮箱。这份周报需要读代码仓库的提交记录、读任务系统里已完成的条目、把两边对上、写成人能看的文字、然后发出去。

为什么不能直接问 AI

你可能会想：这不就是把资料丢给 ChatGPT，让它写一篇吗？问题在于这件事其实有五个独立的步骤，而且每一步都可能出岔子：

1. 去代码仓库拉上周的提交记录——需要凭据，可能拉失败，可能一条都没有
2. 去任务系统拉已完成的任务——另一套凭据，另一套接口
3. 把两边对上：哪个提交对应哪个任务——这一步需要判断力，适合交给 AI
4. 写成周报——也交给 AI，但要求的风格跟上一步完全不同
5. 发邮件——纯机械操作，绝不能让 AI 自由发挥

如果全塞进一次对话，你会遇到三类麻烦。第一，你不知道它错在哪。周报里少了一个项目，是因为仓库没拉到、还是因为 AI 对不上、还是因为它写漏了？一次对话是个黑盒，只能重来。

第二，你没法只重跑一步。第一步拉了三分钟的数据，第四步写崩了，你得从头再来一遍，包括那三分钟。

第三，有些步骤根本不该交给 AI。「发邮件」这一步，你要的是「按这个模板发给这个地址」，不是「AI 觉得应该发给谁」。让 AI 决定收件人，早晚出事。

不这样会出什么事

把一个多步骤任务塞进一次 AI 对话，本质上是放弃了「知道它错在哪」和「只重试出错的那一步」这两个能力。任务简单时无所谓，一旦步骤多起来、或者要每周自动跑，这两个能力就是刚需。

所以系统做的事是：把步骤画出来

AgentLoom 让你在网页上画一张图：每个步骤是一个方块，方块之间用线连起来表示先后顺序。上面那个周报任务画出来大概是五个方块串成一条线，其中第三、第四个方块里放的是 AI，第一、第二、第五个方块里放的是固定动作。

画完之后点「运行」，系统就按图上的顺序一个一个执行。每个方块执行完，它的产出会顺着线传给下一个方块。关键是：每个方块的执行状态、输入、输出，系统都单独记下来。所以出错时你能看到「第三个方块失败了，它当时收到的输入是这些」，而不是只看到「任务失败」。

这张图有个正式名字

工作流 Workflow

就是你画的那张图，加上图上每个方块的配置。一条工作流描述的是「这件事分几步、每步干什么、谁在谁之后」。它是一份定义，本身不会自己动，要有人点运行才会执行。

系统里所有能自动化的事情，最终都表现为一条工作流。

有向无环图 DAG

这是那张图在技术上的名字，听起来吓人，拆开看很简单。有向就是线有箭头，A 指向 B 表示 B 要等 A 干完。无环就是不允许绕回去形成圈——如果 A 等 B、B 等 C、C 又等 A，那三个谁都动不了，系统会永远卡住。所以系统在你连线的时候就会拦掉这种连法。

你可以直接理解成「菜谱的步骤表」：切菜在炒菜之前，炒菜在装盘之前，不可能出现「装盘在切菜之前」这种循环。

为什么还要有「智能体」这个概念

上面的例子里，第三、第四个方块放的是 AI。但如果只是「把提示词发给模型、拿回结果」，那这个方块跟一个普通函数没区别。实际需求比这复杂：

比如第三步「把提交记录和任务对上」，AI 可能需要反复查——先看提交记录，发现某条看不懂，再回头查任务系统里的详细描述，甚至需要打开某个文件看一眼。这不是一次问答，是一个来回若干轮、每轮可能调用工具的过程。

智能体 Agent

一个有持续身份的 AI 角色。它跟「问一次 AI」的区别有四点：它**有自己的一套工具**（能读文件、能查数据库、能跑命令）；它能**多轮自主行动**（自己决定下一步调哪个工具，不用每步都问你）；它有**记忆**（上次的结论下次还能想起来）；它有**版本**（你改了它的配置，旧版本还留着）。

在这个系统里，智能体既可以单独使用（像跟一个专家聊天），也可以塞进工作流当成一个方块用。

这里有个容易搞混的地方值得强调：工作流和智能体是两个平级的概念，不是一个套在另一个里面。你可以只用智能体，不画任何工作流；也可以画一条工作流，里面一个 AI 都没有。它们各自有独立的定义、版本、执行记录。智能体能被塞进工作流当方块用，只是它们之间的一种连接方式。

到这里，系统的核心职责就清楚了

它要做四件事，难度递增：

职责	难在哪
让你把步骤画出来	不算难。难的是要在你连线的时候就拦住不合法的连法，而不是等运行时才报错
按图的顺序执行	要处理分支、循环、嵌套，还要在中途挂掉时能说清挂在哪一步
让 AI 真的能干活	AI 要能跑命令、装依赖、改文件——这意味着要给它一台真机器，还得保证它跑不出去
让很多组织同时用	数据不能串、资源要有上限、行为要留痕

后面所有章节，都是在讲这四件事各自是怎么实现的。

第 2 章

把它想象成一间餐厅

这一章不含任何新知识，只是给你一套对应关系。后面每一章讲到某一块时，你都可以回来对一眼它在餐厅里对应什么。

整体对应关系

顾客在门口的电子菜单上**自己设计一道菜的做法**（画工作流），点确认下单。前台接单并检查这单能不能做（服务器接请求）。接下来单子不是直接进后厨，而是先**挂到叫号板上**（进队列）。后厨的厨师按叫号板上的顺序取单开工（工作进程），需要动火的工序在**独立的封闭灶台**里做（沙箱）。所有食材、半成品、成品分别放在**四个不同的仓库**里（四种数据库）。整间餐厅同时服务多个包间，包间之间**互相看不到彼此的单子**（多租户隔离）。

逐项对照表

餐厅里的东西	系统里对应	后面第几章讲
电子菜单，顾客在上面拖拽设计做法	画布 / Studio	第 6 章
前台，接单、验单、告诉你能不能做	服务器 / Server	第 7 章
叫号板，单子先挂上去再被取走	队列 / BullMQ	第 8 章
厨师，看单子自己决定先切还是先煮	智能体 / Agent	第 9 章
封闭灶台，一人一个，互不串味也烧不到别人	沙箱 / Firecracker	第 10 章
四个仓库：常温、冷藏、调料架、大件储物	四种数据库	第 11 章
包间，A 桌看不到 B 桌点了什么	多租户隔离	第 21 章
后厨统一的术语手册，前台和厨房不能各叫一套名	契约层 / Zod	第 13、19 章

这个类比在哪里会失效

类比总有边界，提前说清楚，免得后面被误导：

- 厨师不会自己改菜谱，但智能体可以。系统里的智能体能提出「我觉得我的配置该这么改」，经你批准后生效。餐厅里没有对应物。
- 餐厅的灶台是固定的，沙箱是用完就销毁的。每次执行可以起一个全新的沙箱，跑完就整个扔掉，下次再起一个干净的。
- 餐厅只有一个后厨，这个系统的「后厨」可以有很多份同时跑。叫号板的意义正在于此：多个厨师从同一块板上取单，谁空谁取。

接下来进入名词表。如果你想先看整体流程，可以直接跳到第 18 章，那一章会把「点一次运行」从头到尾走一遍；名词表随时可以回来查。

第 二 部 分

名词表

后面会用到的技术名词，集中在这里解释一遍。不用背，读到不认识的名词回来查就行。

第 3 章

编程语言与运行环境

这个项目用了五种语言。看起来很杂，但每种都有它非用不可的理由——后面第 12 和第 17 章会讲为什么。这里先认个脸。

JavaScript JS

浏览器唯一能直接执行的语言。你在网页上点按钮有反应，背后就是它。它诞生得很早、设计上有不少历史包袱，最大的问题是不检查类型——你可以把一个数字塞进一个本该放文字的地方，它不会报错，直到运行到某一行才崩。

TypeScript TS

给 JavaScript 加上类型检查的版本。你写代码时声明「这个变量是文字」「这个函数要收两个数字」，然后有个工具在你写完之后、真正运行之前先检查一遍，对不上就报错。

关键是：它检查完之后，类型信息就被丢掉了，真正跑起来的还是普通 JavaScript。这一点很重要，第 13 章讲 Zod 时会回到这里。

这个项目的网页端、服务器端、契约层、插件工具都是 TypeScript 写的，是绝对的主力语言。

Node.js Node

让 JavaScript 能脱离浏览器、在服务器上跑起来的运行环境。有了它，前端和后端才可能用同一种语言。这个项目的服务器就跑在 Node 上。

Rust

一种以「快」和「安全」著称的语言。它的特点是编译器极其严格，很多其他语言要等运行时才暴露的错误，它在编译阶段就拦住。代价是学习曲线陡、写起来慢。

这个项目里它只负责一件事：判断画布上两个方块能不能连线。第 17 章会讲为什么这件小事值得单独用一种语言。

Go

另一种偏底层的语言，擅长跟操作系统打交道——管理进程、配置网络、控制虚拟机这类活。语法简单，编译出来是单个可执行文件，部署方便。

这个项目里它负责管理沙箱：起虚拟机、配网卡、执行防火墙规则。这些活用 JavaScript 干很别扭，Go 是对口的选择。

Dart 与 Flutter

Dart 是语言，Flutter 是基于它的界面框架。它俩的卖点是一套代码同时生成 iOS 和安卓两个 App，不用把界面写两遍。这个项目的手机端用的是它。

SQL

跟数据库对话的专用语言。「把所有姓张的用户查出来」这种要求，写成 SQL 大概是 `SELECT * FROM users WHERE name LIKE '张%'`。它不是通用编程语言，只用来查数据、改数据。

WebAssembly WASM

一种能在浏览器里运行的、接近机器码的格式。它的意义是：让 Rust、C++ 这类语言写的程序也能在网页里跑，而不是只能用 JavaScript。

这个项目把 Rust 写的连线判断编译成 WASM，塞进浏览器执行。

第 4 章

框架与工具库

「框架」和「库」的区别用一句话讲：库是你调用它，框架是它调用你。用库像用工具箱，你决定什么时候拿哪把螺丝刀；用框架像进流水线，框架规定了流程，你往里填自己的部分。

服务器那一侧

NestJS Nest

服务器端的框架。它规定了代码该怎么组织：每个业务领域一个「模块」，模块里分「控制器」（负责接收网络请求）、「服务」（负责真正干活）。

它最有用的一点叫依赖注入——你的代码不需要自己去创建它依赖的东西，只要声明「我需要一个数据库连接」，框架会把它送进来。好处是测试时可以轻易换成假的。

这个项目的服务器有 41 个这样的模块。

Fastify

负责处理网络请求的底层组件——接收 HTTP 请求、解析、返回响应。它的竞争对手叫 Express，是老牌选择。选 Fastify 的理由是它更快，代价是生态没 Express 那么大。

它跟 NestJS 是配合关系：NestJS 管代码组织，Fastify 管收发请求。

Zod

一个用来检查数据长得对不对的库。你先写一份「规格说明」，比如「这个东西必须有 name 字段且是文字、有 age 字段且是正整数」，然后拿真实数据去对，不合规就报错并告诉你哪个字段错了。

它在这个项目里的地位特别高，单独占了第 13 章。

Drizzle

让你用 TypeScript 写数据库查询，而不用手写 SQL 字符串。它属于「对象关系映射」这类工具（简称 ORM），竞争对手是 Prisma、TypeORM。

它的特点是「薄」——生成的 SQL 跟你写的代码几乎一一对应，不做太多幕后魔法。第 15 章讲为什么这一点重要。

BullMQ

队列工具，也就是餐厅里的「叫号板」。它让服务器可以把耗时的活先记下来、稍后由别的进程慢慢干，而不是让发请求的人一直等着。

它还负责定时任务（每周一早上八点跑一次）和失败重试。

Socket.IO

让服务器能主动给网页推消息的工具。普通的网页交互是「网页问一句、服务器答一句」，服务器没法主动说话。有了它，执行进度可以实时推到你的屏幕上，不用你不停刷新。

它还能在断网重连后补上你错过的消息，这一点第 18 章会讲。

网页那一侧

React

目前最主流的网页界面框架。它的核心思想是把界面切成一个个「组件」——一个按钮是组件，一个卡片是组件，整个页面由组件拼起来。数据变了，组件自动重绘。

React Flow

专门做「拖拽画流程图」的库。方块能拖、线能连、画布能缩放，这些都是它提供的。整个画布界面的地基就是它。

Vite

开发和打包工具。你改一行代码，它负责让浏览器几乎立刻看到效果；发布时它把几百个文件压缩合并成浏览器能高效加载的形式。

TanStack Query

管理「从服务器拿来的数据」的库。它解决的问题很具体：同一份数据被三个组件用到，不该请求三次；数据可能过期，要知道何时重新拿；请求失败要能重试。这些它都替你办了。

Zustand

管理「纯本地状态」的库——比如画布上当前选中了哪个方块、侧边栏是展开还是收起。这些数据服务器不关心，只存在于你这一次的浏览过程里。

这个项目有条明确规矩：服务器来的数据交给 TanStack Query，纯本地的交给 Zustand，不许混。

Tailwind CSS

写样式的方式。传统做法是另开一个文件写「这个按钮红色、圆角、内边距 8 像素」，Tailwind 让你直接在元素上写一串短标记达到同样效果。好处是不用在两个文件之间来回跳。

第 5 章

存数据的四个地方

这个系统同时用了四种数据库。听起来是过度设计，其实每一种存的东西性质完全不同——第 11 章会详细讲为什么不能合成一个。这里先认识它们。

PostgreSQL Postgres / PG

主数据库，存所有「正式记录」：用户、组织、工作流定义、执行记录、审计日志。它是关系型数据库，意思是数据存成一张张有固定列的表，表之间能互相引用。

它最重要的能力是事务——一组操作要么全部成功，要么全部撤销，不会停在中间状态。转账时「A 减钱」和「B 加钱」必须同生共死，靠的就是这个。

Redis

把数据放在内存里的数据库，所以极快，但容量小、断电就没。它在这个项目里干两件事：做队列的底座（叫号板本身就存在 Redis 里），以及存计数器（某个组织今天调了多少次接口）。

这些数据的特点是「短命、高频、丢了也不致命」，正好是 Redis 的强项。

Qdrant

向量数据库。这个概念稍绕，但值得理解，因为它是「按意思检索」的技术基础。

普通数据库只能精确匹配：你搜「苹果」就只找到含「苹果」的记录。向量数据库能按意思找：你搜「水果」，它能把「苹果」「香蕉」都翻出来，因为它存的不是文字本身，而是文字经 AI 转换成的一串数字（叫「向量」），意思相近的向量在数学上距离也近。

它在这个项目里有两个用户：知识库检索（把文档切段、转成向量存起来，提问时找最相关的几段塞给 AI）和智能路由的学习（把历史调用的特征存成向量，用最近邻和小型神经网络来预测哪个模型更合适）。

要特别注意：智能体的记忆不用它，那一块走的是另一条路，下面一条会讲。

全文检索 Full-text Search

PostgreSQL 自带的一种检索能力，介于「精确匹配」和「按意思匹配」之间。它把一段文字拆成词、去掉「的」「是」这类没信息量的词、把词变成统一形式（比如 running 归到 run），然后按关键词命中的多少和位置算一个相关度分数排序。

它比精确匹配聪明（搜「部署」能命中「部署方案」「如何部署」），但比向量检索笨（搜「上线」找不到「部署」，因为它们没有共同的词）。好处是不需要额外的数据库，也不需要调用 AI 把文字转成向量——省一次网络请求、省一笔费用。

智能体的记忆检索用的就是它。

MinIO

对象存储，可以理解成「自己搭的网盘」。存的是文件本身：用户上传的附件、技能说明文档、插件安装包。

为什么不直接把文件塞进主数据库？因为关系型数据库擅长处理「大量小记录」，不擅长「少量大文件」，硬塞会拖慢整个库。

几个反复出现的概念

API 应用程序接口

两个程序之间说话的约定。网页想拿到工作流列表，就按约定向服务器的某个地址发一个请求，服务器按约定返回一段数据。这份约定就是 API。

约定的形式包括：请求发到哪个地址、要带哪些参数、返回的数据长什么样。三个端（网页、手机、服务器）对这份约定的理解必须完全一致，否则就会出现「服务器说这个字段可能为空、网页以为它一定有值」这种事故。第 19 章专门讲怎么防这个。

多租户 Multi-tenancy

一套系统同时服务多个互不相关的组织，而每个组织只能看到自己的数据。「租户」就是一个组织。

难点在于：所有组织的数据其实存在同一个数据库的同一张表里，靠代码里的过滤条件区分。一旦某个查询忘了加过滤条件，就会把别人的数据返回出去。第 21 章讲这个系统怎么防。

沙箱 Sandbox

一个被隔离起来的执行环境。让 AI 在里面跑命令、改文件，它干什么都出不了这个环境的边界，也影响不到外面。

为什么必须有：因为 AI 生成的代码是不可信的——不一定是恶意，可能只是写错了一个删除命令。第 10 章和第 20 章讲实现。

客户端持钥加密 常被叫作 E2EE

这套机制在项目里叫「端到端加密」，但它跟聊天软件那种端到端加密不是一回事。真实流程是三步：一、你在浏览器里生成一对密钥，**只上传公钥**，私钥留在本地。二、AI 的输出在服务器的的工作进程里产生，服务器**用你的公钥**加密后才写进数据库。三、你查看时，浏览器用本地私钥解开。

所以准确的名字是「客户端持钥的静态数据加密」：钥匙只在你手里，数据库拖走读不出；**但明文在服务器的的工作进程里存在过那一瞬间**——因为加密是在那里做的，浏览器侧只有解密能力，没有加密上传的路径。

它防的是数据库被偷、备份泄露、运维越权查表；不防服务器运行时被攻破。算法是 RSA-4096 包一把一次性的 AES-256 密钥。

第 三 部 分

六个组成部分

把系统拆成六块，每块讲清三件事：它干什么、不这么做会怎样、它实际上是怎么运作的。

第 6 章

画布：你看到的那个界面

画布是整个系统唯一被用户直接看到的部分。它的职责听起来简单——让人拖方块、连线——但有一个要求让它变难：不合法的连法必须在你松手之前就被拦住。

为什么不能等运行时再报错

设想你画了一条二十个方块的工作流，点运行，跑到第十七个方块才报错「这里收到的数据类型不对」。此时前十六步已经花了几分钟、可能还调用了付费的 AI 接口。你改完再跑，又要从头等一遍。

不这样会出什么事

如果连线时不检查，错误就会推迟到运行时才暴露。而运行时的错误代价高得多：浪费时间、浪费调用额度，而且你要靠日志倒推「到底是哪两个方块接错了」。检查越早，代价越低——这是整个系统里反复出现的一条原则。

那怎么判断「能不能连」

每个方块有若干接口：左边是输入口，右边是输出口。每个口有类型标签，一共 14 种，比如「文本」「图片」「模型」「工具」「沙箱」。

打个比方

就像插头和插座。两孔插头插不进三孔插座，这件事在你还没用力的时候就已经确定了，不需要真的通电试一次。类型标签的作用就是让系统提前知道「这两个口形状不匹配」。

但实际情况比插头复杂，因为有些类型之间是可以转换的。比如一个方块输出「文本」，下一个方块要「结构化数据」——这两个不一样，但中间加一步解析就能接上。所以判断结果不是简单的能/不能，而是四档：

判断结果	含义
完全兼容	直接连，什么都不用做
需要转换	能连，但系统会在中间插一步转换。目前只允许三种转换：文本转结构化数据、结构化数据转文本、技能转文本
部分兼容	两边都是结构化数据，但字段对不齐。系统会列出缺哪些字段，并猜测你可能想怎么对应——它比较字段名的相似度，给出「你大概是想把 <code>userName</code> 接到 <code>user_name</code> 」这类建议
不兼容	拒绝连线，并说明原因

「需要转换」只允许三种，是有意为之。每多一条自动转换，用户就多一次「它居然偷偷帮我转了」的意外。宁可让用户自己显式加一个转换方块，也不要让系统悄悄做决定。

这个判断实际上跑在哪

这里有个值得知道的细节。判断分两步，而且两步是不同的技术：

1. 结构性检查，用 TypeScript 写，跑在浏览器主线程：这两个口是否存在、是不是自己连自己、目标口是否已经连满了上限。这些检查很快，不涉及复杂计算。
2. 类型与结构比较，用 Rust 写、编译成 WASM，跑在后台线程：递归比较两边的数据结构，算出上面那四档结论和字段映射建议。

放在后台线程的原因是：这个计算在你拖动鼠标的过程中会被反复触发，如果占用主线程，画布会卡顿。第 17 章会讲为什么这部分用 Rust，以及这个选择目前有个没兑现的地方。

画布之外，这个界面还管什么

除了画布，网页端还包括智能体配置页、执行历史、知识库管理、插件市场、监控面板、组织与成员管理等。这些页面技术上没什么特别，都是常规的「列表加表单」。整个网页端的代码按功能切成一个个独立的文件夹，规定跨文件夹只能通过对外统一出口互相引用，不许直接伸手进别人内部——这条规矩由工具自动检查，违反了就报错。

第 7 章

服务器：接单和派活的那一层

服务器是系统的中枢。它不干重活，它的职责是判断这个请求该不该做、然后把活派出去。真正的重活在别的进程里。

一个请求进来，要过五道关

任何一个网络请求到达服务器后，在真正被处理之前，要依次通过五道检查。顺序是固定的，而且顺序本身有讲究：

序	这一关做什么	为什么排在这个位置
1	初步解析请求，试着看出它属于哪个组织	此时还没验证身份真假，所以这一步的结论只能用于粗略分类，不能用于任何数据访问决策
2	限流：这个组织最近请求太频繁了吗？今天的额度用完了吗？	排在验身份之前是为了尽早拒绝洪水般的请求。但这个顺序目前带来了一个真实的问题，第 23 章会讲
3	验证身份：这个请求的凭据是真的吗？	这一步才真正确认「你是谁」。通过之后，可信的身份信息才被写进请求里
4	组织准入：你确实属于这个组织吗？	身份确认之后才能做这一步
5	角色权限：你的角色允许做这个操作吗？	最后一道，管的是「能不能删」这类细粒度权限

五道关都过了之后，还有一步：系统把接下来所有的数据库操作**包进一个事务**，并在事务里设置好「当前是哪个组织」。这一步是数据隔离的关键，第 21 章详讲。

为什么服务器不干重活

因为一次 AI 工作流可能跑几分钟甚至几十分钟。如果服务器接到请求就开始干、干完才回复，会出现三个问题：

- 用户的浏览器等不了那么久，网络连接会超时断开
- 服务器的处理能力会被占满——同时来十个长任务，第十一个人连页面都打不开
- 服务器一重启，正在跑的活全丢了，因为它只存在于内存里

所以服务器的做法是：把「要干这件事」这个意图写进数据库，然后往队列里放一张单子，立刻回复用户「收到了，编号是 XXX」。真正的执行由另一批进程负责。这就是下一章的内容。

第 8 章

队列：为什么不能「来了就干」

队列是这个系统里最容易被低估的一块。它看起来只是个待办清单，实际上它解决的是「怎么让长任务可靠」这个问题。

打个比方

餐厅点单之后，单子不会直接塞给某个厨师，而是打印出来挂到叫号板上。好处是三个：哪个厨师空了自己来取，不会出现一个人忙死另一个人闲着；某个厨师突然请假，他手上没做完的单子还挂在板上，别人能接手；单子上写着做了几次，同一单反复失败就不再重发，转去人工处理。

三个好处，对应三个技术能力

一、削峰

同时来一百个执行请求，系统不会试图同时跑一百个。队列让它们排着，由固定数量的工作进程按能力取。结果是系统在过载时会变慢，但不会崩。这个区别很关键：变慢是可恢复的，崩了要人工介入。

二、可靠

工作进程取走一张单子、干到一半时机器重启了，这张单子不会消失——它还在队列里，只是被标记为「有人在处理」。超过设定时间没有完成确认，队列会把它重新交给别人。

不这样会出什么事

如果任务只存在于某个进程的内存里，那个进程一挂，任务就凭空消失了。用户看到的是「我明明点了运行，但什么都没发生，也没有报错」——这是最难排查的一类故障，因为连日志都没有。

三、定时与重试

「每周一早上八点跑一次」这种需求，交给队列自带的定时功能，不需要自己写一个循环去数时间。失败重试也是队列内置的：第一次失败等 1 秒重试，第二次等 4 秒，第三次等 16 秒——间隔越来越长，避免在服务方已经挂了的情况下疯狂重试把它压得更死。

一个必须注意的顺序问题

服务器要做两件事：把执行记录写进数据库、往队列里放单子。这两件事的顺序不能颠倒，而且中间不能出岔子。

如果先放队列再写数据库：工作进程可能在数据库写完之前就取到了单子，然后发现「这个执行记录不存在」，直接失败。这种问题的表现是随机的——机器快的时候没事、慢的时候报错，极难排查。

所以正确做法是：先在事务里写完数据库，确认提交成功之后，才往队列里放单子。这样工作进程取到单子时，记录一定已经存在。

第 9 章

智能体：跟直接问 AI 差在哪

第 1 章说过智能体有四个特点：有工具、能多轮、有记忆、有版本。这一章逐个讲它们各自解决什么问题。

一、有工具：AI 能真的动手

普通对话里，AI 只能输出文字。你问它「这个项目有多少个文件」，它只能猜。给它配上工具之后，它可以真的去数。

工作方式是这样：系统告诉 AI「你有这些工具可用，每个工具需要什么参数」。AI 在回答过程中如果需要，就输出一个「我要调用某工具、参数是这些」的请求；系统真正执行这个工具，把结果送回给 AI；AI 拿到结果继续思考。这个来回可能进行很多轮。

二、能多轮：不用每一步都问你

这是智能体和普通对话最大的区别。你交给它一个目标，它自己决定分几步、每步用哪个工具，直到目标达成或者卡住。

但「自主」是有边界的。有些操作有实际后果——写文件、改配置、删东西。这类操作系统会停下来问你：

工具授权

AI 想执行有后果的操作时，任务会挂起并弹出确认。你可以本次批准、本次拒绝，或者「这轮会话里以后都同意」。

重点是：挂起期间任务不算失败，它只是在等。如果你一直不理，超时后按预设策略处理——自动同意、自动拒绝、或者升级给你的上级，最多升级三次。

三、有记忆：一张存在主数据库里的图

对话结束后，其中值得长期记住的结论会被抽出来，存成一张图——不是一堆散乱的笔记，而是带关系的网络：节点是一条条结论，边表示它们之间的关联（「项目 A」连到「张三负责」连到「用了 Redis」）。节点和边都存在 PostgreSQL 里，各自是一张表。

下次对话时，系统按当前话题去这张图里检索相关的部分，塞进 AI 的上下文。这里有个容易想错的地方值得说清楚：这一步用的不是向量数据库，而是 PostgreSQL 自带的全文检索。

具体做法是把查询词清洗一遍（去掉可能破坏语法的特殊符号，多个词之间按「都要满足」组合），然后用数据库的排序函数算相关度，再让数据库直接生成带高亮标记的摘要片段返回。

为什么记忆用全文检索，知识库却用向量库

两者要检索的东西性质不同。**记忆**是系统自己写下的短结论，用词由系统控制、比较规范，关键词匹配足够用；而且记忆检索发生在每次对话开始时，走全文检索不需要调用 AI 把查询转成向量，省一次网络往返和一笔费用。

知识库装的是用户上传的文档，用词五花八门——用户问「怎么上线」，文档里写的是「部署流程」，两者没有共同的词，只有向量检索能对上。这种场景多花一次转换是值得的。

四、有版本：改配置不会丢历史

你调整了一个智能体的提示词、换了模型、加了工具，这些改动会形成一个新版本，旧版本仍然保留。这意味着：出问题可以回退，而且能对比「上个版本表现更好还是这个」。

另外两个值得知道的机制

技能 Skill

一份用 Markdown 写的操作手册，存在文件存储里。比如「怎么给这个项目写提交信息」「公司的代码规范是什么」。

巧妙之处在于加载方式：平时只把「有哪些技能可用」的清单塞进 AI 的上下文，正文不塞。AI 判断需要某份手册时才去取正文。这样几十份技能也不会挤占上下文空间。相当于随身带着工具书的目录，需要时才翻开某一页。

智能路由

决定这次调用用哪个 AI 模型。策略有好几种：最省钱、最快、质量最好、历史表现最好，以及回退链——按顺序试，这个模型挂了自动换下一个。

默认是回退链，因为它对线上最友好。这里有个细节值得注意：只有「非认证失败」才换模型重试。如果是密钥错了，换个模型也一样错，重试没有意义，只是浪费时间。

第 10 章

沙箱：为什么不敢让 AI 直接跑命令

这是整个系统里安全要求最高的部分。一旦允许 AI 执行命令、装依赖、改文件，问题的性质就变了——从「它答得对不对」变成「它会不会把机器搞坏、会不会读到别人的数据」。

先说清风险是什么

AI 生成的代码是不可信输入。这个判断跟「AI 好不好」无关，它是个结构性事实：代码是模型现场生成的，没有人逐行审过就直接执行了。

风险不一定来自恶意。更常见的是普通错误——一个路径写错的删除命令、一个把整个磁盘写满的循环、一个不小心读到了配置文件里的密钥。如果这些直接跑在服务器上，后果是真实的。

为什么不用 Docker

这是个内行会追问的点，值得讲清楚。

容器 Container / Docker

一种轻量隔离技术。它让程序以为自己独占一台机器，实际上是跟别人共享的。启动很快，开销很小，是目前部署应用的主流方式。

但它的隔离有个根本限制：所有容器共享宿主机的操作系统内核。内核是操作系统最核心的那一层。如果内核有漏洞，一个容器里的程序有可能突破边界，影响到宿主机和其他容器。

微型虚拟机 Firecracker

每个沙箱是一台拥有自己独立内核的虚拟机。因为内核不共享，容器那类「突破边界」的路径就被堵住了。传统虚拟机的问题是笨重——启动要几十秒，占用资源大。Firecracker 是专门为「大量、短命、快速启动」场景做的，启动时间在毫秒级，资源开销接近容器。

代价是：要自己管虚拟机的生命周期、网络配置、磁盘挂载。这比敲一行 `docker run` 复杂一个量级。这部分复杂度就是用 Go 写的那个组件在承担。

打个比方

容器像同一栋楼里的不同房间——隔音、有门锁，但共用一套水电和承重结构。微型虚拟机像各自独立的小屋，自带水电。前者盖起来快、成本低；后者贵一点，但一间出问题不会影响另一间的地基。

跑自己写的、审过的代码，用房间够了。跑不知道会干什么的代码，值得盖独立小屋。

笼子上的五道闸

光有虚拟机不够。虚拟机里的程序仍然能读写文件、发网络请求，所以还要在五个方向上各加一道闸。第 20 章会讲每一道的具体实现，这里先给出全貌：

方向	限制的内容
读文件	只允许碰两个指定目录，而且要防「用软链接绕出去」这种花招
写文件	比读更严：不许把根目录当文件写，已存在的目标必须是普通文件
解压缩包	压缩包里的路径也是攻击面——一个精心构造的包能把文件写到目录外面去
网络	只放行「用自己被分配的地址」发出的包，其他一律丢弃；禁止访问内网和其他沙箱
凭据	沙箱内不持有能访问应用的证书，所有回调必须经过一个受控的中转站

五道闸有一个共同的设计原则：**默认拒绝，明确放行**。不是列出「不允许什么」，而是列出「只允许什么」，剩下的全拒。这两种写法的区别在于：前者漏写一条就是一个漏洞，后者漏写一条只是少了一个功能。

第 11 章

为什么要四种数据库而不是一种

「用四种数据库」听起来像技术炫耀。实际上如果只用一种，会在四个不同的地方各撞一次墙。

四类数据的性质完全不同

数据	性质	只用 PostgreSQL 会怎样
业务记录	要求绝对不能丢、要事务、要能复杂查询	这就是它的主场，没问题
队列与计数器	极高频、短命、丢了不致命	每次取单子都要读写磁盘，几百次每秒就把主库拖垮了。而主库一慢，整个系统都慢
语义检索	要按「意思相近」排序，不是精确匹配	PostgreSQL 有向量插件，小规模够用；但数据量大了之后检索性能和调优手段都不如专用的
文件	单个几 MB 到几百 MB，只按名字整个取出	大字段会撑爆数据表、拖慢备份、让每次全表操作都变得昂贵

打个比方

餐厅不会把所有东西放一个仓库：[常温货架](#)放米面（业务记录，稳定、要清点）；[灶边的调料台](#)放随手要用的（队列计数器，高频存取，就近放）；[冷库](#)放需要特殊条件的（向量数据，需要专门的检索能力）；[后院大件储物间](#)放整箱整袋的（文件）。

硬要合成一个，结果是：为了照顾大件把货架做得很深，导致拿一包盐要走十米。

代价也要说清楚

四种数据库不是免费的。它带来三个实实在在的负担：

- 运维复杂度：四套东西都要部署、监控、备份、升级
- 没有跨库事务：往 PostgreSQL 写记录和往 MinIO 上传文件不能保证同生共死，必须在代码里自己处理「一边成了一边没成」的情况
- 本地开发门槛高：新人要把四个服务都跑起来才能开始干活

这个项目对第三点的处理是：开发环境只用容器起 Qdrant 一个，其余三个要么连外部实例、要么用一套现成的整合栈。

第 四 部 分

为什么用这些技术

每一节讲三件事：这个技术是什么、不用它会撞到什么墙、为什么选它而不是同类的竞争对手。

第 12 章

为什么整个项目用 TypeScript

这是最基础的一个选择，也是后面很多选择的前提。

先说 JavaScript 的问题

JavaScript 不检查类型。下面这段代码完全合法，能正常保存、正常部署：

```
function 打折(价格) {  
  return 价格 * 0.8;  
}  
  
打折(100)      // 得到 80，正确  
打折("100")   // 得到 80，居然也对（悄悄把文字转成了数字）  
打折("一百")  // 得到 NaN，意思是「不是数字」  
打折()        // 也得到 NaN
```

最后两种情况不会报错，只是算出一个叫 **NaN** 的怪东西，然后它会继续往下传，可能一路传到数据库里、传到用户的账单上，直到某个完全无关的地方突然崩掉。而那个崩掉的地方，跟真正的错误来源可能隔着十几层调用。

不这样会出什么事

这类错误的特征是**报错的地方不是出错的地方**。你花两小时排查一个显示异常，最后发现是三天前另一个人在完全不相干的模块里少传了一个参数。项目越大，这个成本增长得越快。

TypeScript 怎么解决

```
function 打折(价格: number): number {  
  return 价格 * 0.8;  
}  
  
打折("100")    // 检查器直接报错，代码根本编译不过去  
打折()         // 同样报错：缺少参数
```

关键收益不只是「少出错」，而是错误被提前到了写代码的时候。你在编辑器里就能看到红线，不需要等部署、不需要等用户投诉。

为什么要求前后端都用它

因为可以共享类型定义。服务器定义「工作流长这样」，网页端直接引用同一份。服务器改了一个字段，网页端立刻编译不过——错误在几秒内暴露，而不是等到上线后某个页面白屏。

如果前后端用不同语言，这份定义就必须在两边各写一份，靠人力保持同步。而人力同步的东西，一定会在某个时刻不同步。第 19 章整章都在讲这件事。

它的一个重要局限，直接引出下一章

TypeScript 的检查只在编译期有效。检查完之后，所有类型信息都被删掉，真正跑起来的还是普通 JavaScript。

这意味着：如果数据是运行时才从外面进来的——用户提交的表单、别的系统发来的请求、数据库读出来的记录——TypeScript 一点忙都帮不上。你可以声明「我期望收到一个有 name 字段的对象」，但如果对方实际发来的是一段乱码，程序照样接受，然后在某个地方崩掉。

打个比方

TypeScript 像装修时的图纸审查——它能确保设计上门宽匹配门框。但它管不了[装修完成之后，谁从门进来](#)。要在门口验证来客身份，需要另一个东西：门卫。

Zod 就是那个门卫。

第 13 章

Zod 是什么，为什么它这么重要

这一章最长，因为 Zod 在这个项目里不只是一个工具库，它是整套跨端约定的地基。理解了它，第 19 章会变得很容易。

问题：外面进来的数据，没人检查

服务器收到一个创建智能体的请求，理想情况下数据长这样：


```
{ "name": "周报助手", "runtimeMode": "sandbox" }
```

但实际可能收到任何东西：字段名拼错、少了必填项、类型不对，或者干脆是一段恶意构造的内容。TypeScript 挡不住任何一种，因为这些都发生在运行时。

所以必须在代码里手写检查。手写版大概是这样：

```
if (typeof body.name !== 'string' || body.name.length === 0) {
  throw new Error('name 必须是非空文字');
}
if (body.runtimeMode !== 'sandbox' && body.runtimeMode !== 'no_sandbox') {
  throw new Error('runtimeMode 取值不对');
}
// .....每个字段都要来一遍
```

这么写有三个问题。一是啰嗦，几十个接口每个都要写一大段。二是容易漏，加了新字段忘了加检查。三是最要命的——它和类型声明是两份独立的东西，可以不一致：

```
type CreateAgent = { name: string; runtimeMode: 'sandbox' | 'no_sandbox' };
// 类型说 runtimeMode 有两种取值

if (body.runtimeMode !== 'sandbox') { throw ... }
// 检查却只认一种。两边对不上，没有任何工具会提醒你
```

不这样会出什么事

类型声明和实际检查各写一份，就一定会在某次修改后不一致。而且这种不一致没有任何工具能自动发现——类型检查器只看类型声明，测试只测检查逻辑，两边都觉得自己是对的。最终表现为线上偶发的奇怪数据。

Zod 的做法：只写一份，类型自动推导出来

```
const CreateAgentSchema = z.object({
  name: z.string().min(1),
  runtimeMode: z.enum(['sandbox', 'no_sandbox']),
});
```

这一份东西同时给你两样：

```
// 一、运行时的检查器
CreateAgentSchema.parse(收到的数据)
// 合规就返回数据；不合规就抛错，并说明哪个字段错在哪

// 二、静态类型，从上面那份规格「反推」出来
type CreateAgent = z.infer<typeof CreateAgentSchema>;
// 等价于 { name: string; runtimeMode: 'sandbox' | 'no_sandbox' }
```

这里是整章的关键：类型不是你另写的，是从检查器身上推导出来的。所以「类型说三种取值、检查只认两种」这种事在结构上就不可能发生——它们本来就是同一个东西的两个面。

打个比方

过去是：门口贴一张「入场要求」海报（类型声明），门卫另有一本自己的检查手册（校验代码）。海报改了没人通知门卫，于是海报写「凭票入场」，门卫还在只看身份证。

Zod 的做法是：只有门卫的手册是真的，海报直接由手册自动打印。手册改一个字，海报下一秒就跟着变。

项目里真实的用法

规格写好之后，包装成服务器框架认识的形式：

```
export class CreateAgentDefinitionDto
  extends createZodDto(CreateAgentDefinitionSchema) {}
```

然后在接收请求的地方挂上检查：

```
async create(
  @Body(new ZodValidationPipe(CreateAgentDefinitionSchema))
  dto: CreateAgentDefinitionDto, // 这里只用它的类型
) { ... }
```

意思是：这个接口的请求体，进来之前先用那份规格检查一遍。通过了，`dto` 就是干净可信的数据；没通过，请求根本进不到函数体里。

规格里还能顺手做两件事，这也是手写检查很难做好的：字段名归一（客户端传下划线还是驼峰都接受，统一转成一种）和跨字段校验（比如「选了沙箱模式，就必须同时给出沙箱配置」）。

检查失败时返回什么

这个项目没用默认的错误格式，而是统一转成一种标准化结构，状态码 422（意思是「格式我看懂了，但内容不合规」）：

```
{
  "type": "https://agentloom.dev/errors/validation-error",
  "title": "Validation Error",
  "status": 422,
  "detail": "Request validation failed",
  "errors": [
    { "field": "runtimeMode", "message": "取值必须是 sandbox 或 no_sandbox" }
  ]
}
```

`errors` 里每条是「哪个字段、错在哪」，前端可以直接把消息显示在对应输入框下面。

一个反直觉的坑：请求和响应不能共用一份规格

这一节稍绕，但它是项目里一条明文规矩，也是内行会追问的点。

Zod 支持给字段设默认值。比如分页查询：

```
const ListQuerySchema = z.object({
  page: z.coerce.number().int().min(1).default(1),
  pageSize: z.coerce.number().int().default(20),
});
```

`.default(1)` 的意思是「客户端可以不传 `page`，不传就当 1」。这对请求是完全正确的语义。问题在于服务器返回响应时，这两个字段的语义反过来了：

场景	<code>page</code> 字段的语义
请求	可选——客户端不传也行
响应	必填——服务器一定会给

如果图省事，直接拿请求的规格当响应用，接口文档生成器会按请求的语义去理解，把这些字段标成「可能没有」。于是网页端拿到的类型是「`page` 可能不存在」，前端得写一堆没必要的判空；更糟的是凭空放宽了真实的接口约定——明明服务器保证一定给，文档却说可能没有。

所以规矩是：响应必须另写一份规格，按「输出形状」建模。响应里的 `page` 声明成必填的正整数，不带默认值。

同一个模块里还有个更彻底的例子：详情响应把所有可选字段声明成 `nullable`（可以是空值），而请求侧对应字段是 `optional`（可以不存在）。这两个不一样：

- `optional`：这个键可能根本不在数据里
- `nullable`：这个键一定在，但值可能是空

服务器返回时会显式把缺失字段补成空值，所以响应侧一律是 `nullable`。「请求形状 $\neq$ 响应形状」这句话，落在代码上就是这个区别。

为什么不用 `class-validator`

`class-validator` 是这个领域的老牌选择，写法是在类的属性上加标记：

```
class CreateAgentDto {
  @IsString()
  @MinLength(1)
  name: string;
}
```

看起来清爽，但它有个结构性问题：`@IsString()` 和 `name: string` 是两份独立的声明。你可以把类型改成 `number` 而忘记改标记，没有任何工具会报错。

这就回到本章开头那个问题——两份声明就会不一致。Zod 用「类型从校验器推导」在结构上消灭了这个可能。这是项目里「用 Zod 不用 class-validator」那条规矩的实质理由。

第 14 章

后端为什么用 NestJS 加 Fastify

这两个是配合关系，不是竞争关系。NestJS 管代码怎么组织，Fastify 管请求怎么收发。

NestJS 解决的是「41 个模块怎么不乱」

如果不用框架，一个有 41 个业务领域的服务器很快会变成一团麻：每个人按自己习惯组织代码，文件到处放，模块之间随意互相调用。半年后没人说得清「改这一行会影响什么」。

NestJS 强制一套结构：每个业务领域一个模块，模块内部按职责分文件，后缀固定。这套约定的价值不在于它多聪明，而在于它是统一的——任何人打开任何一个模块，都知道该去哪个文件找什么。

依赖注入是它最实用的能力

传统写法里，一个服务需要数据库连接，就自己去创建一个。问题是测试时你没法换掉它——它写死在代码里了。

依赖注入的做法是：服务只声明「我需要一个数据库连接」，由框架在启动时送进来。测试时框架可以送进一个假的，于是不用真连数据库也能测业务逻辑。

Fastify 替换掉了默认的 Express

NestJS 默认搭配 Express——最老牌、生态最大的选择。这个项目换成 Fastify，理由是吞吐更高。代价是有些为 Express 写的插件不能直接用。

这里有个真实的坑值得一提，因为它说明了「换掉默认选择」的隐性成本：项目锁定了 Fastify 的某个具体版本，但某个上传文件的插件会拉进另一个版本。两份同名但版本不同的东西同时存在，会导致类型系统认为它们不兼容，插件根本注册不上。解决办法是在包管理配置里强制全项目只用一个版本。

打个比方

就像两家供应商送来同规格的螺丝，但螺纹标准差一点。单独看都没问题，装在同一台机器上就是拧不进去。解决办法不是换螺丝，而是规定「全厂只认一家的标准」。

第 15 章

为什么用 Drizzle 碰数据库

这一类工具叫 ORM。它们的共同目标是：让你别手写 SQL 字符串。

手写 SQL 的问题

```
const sql = "SELECT * FROM agents WHERE name = '" + 用户输入 + "'";
```

这行代码有两个毛病。第一，如果用户输入里含引号，SQL 语法就断了——更糟的是攻击者可以借此拼进自己的命令，这叫 SQL 注入，是最经典的安全漏洞之一。第二，字段名写错了没人告诉你，直到运行时数据库报错。

Drizzle 的做法

```
await db.select().from(agents).where(eq(agents.name, 用户输入));
```

`agents.name` 是个真实的 TypeScript 对象，字段名拼错编译就不过。用户输入通过参数传递而不是拼进字符串，注入问题自然消失。

为什么是 Drizzle 而不是 Prisma

Prisma 是这个领域最流行的选择，功能更全、文档更好。这个项目选 Drizzle，核心理由是它薄。

「薄」的意思是：你写的代码和它生成的 SQL 几乎一一对应，看着代码就知道数据库会收到什么查询。Prisma 做了更多封装，某些写法背后会变成好几条查询，性能问题不容易预判。

还有一个具体原因：这个项目需要在事务里下发原生 SQL 语句来设置数据隔离的上下文（第 21 章会讲）。这种「大部分时候用工具、关键处直接写 SQL」的混合用法，薄工具支持得更自然。

代价也要说清楚

薄的代价是要自己写的东西更多，而且 Drizzle 生态比 Prisma 小，遇到问题能搜到的答案少。这是个真实的取舍，不是「Drizzle 全面更好」。

第 16 章

前端那一堆库各自在干什么

网页端用了十几个库，看起来很多。但它们各管一件事，职责边界很清楚。

一条明确的分工线：数据从哪来，就交给谁管

数据类型	交给谁	例子
从服务器拿来的	TanStack Query	工作流列表、智能体详情、执行历史
纯本地的、临时的	Zustand	画布上选中了哪个方块、侧边栏开合
该出现在网址里的	URL 参数	列表的筛选条件、当前页码

不这样会出什么事

如果把服务器数据也塞进本地状态管理，就会出现两份真相：一份在本地，一份在服务器。它们什么时候同步、谁覆盖谁，会变成一团糊。典型症状是「我明明改了，刷新一下又变回去了」。

把筛选条件放进网址的理由更实际：用户能把当前筛选结果的链接发给同事。放在内存里就做不到。

一个容易被忽略的细节：命名风格转换

服务器那边字段叫 `runtime_mode`（下划线分隔），网页端习惯叫 `runtimeMode`（驼峰）。

处理方式是在网络请求的统一出入口做自动转换——发出去时转下划线，收回来时转驼峰。两边各自都用自己习惯的风格，没人需要在业务代码里手动转。

但有个例外要记住：实时推送的消息不做转换，两边都用驼峰。因为那条链路的数据结构由契约包统一定义，没有历史包袱。手机端的模型也明确配置成不转换，跟这条对齐。

第 17 章

为什么会掺进 Rust 和 Go

一个 TypeScript 项目里出现另外两种语言，需要理由。这两个理由不一样，而且其中一个目前只兑现了一半——这一点必须讲清楚。

Go：因为要跟操作系统打交道

沙箱管理要做的事包括：启动虚拟机、配置网卡、下发防火墙规则、挂载磁盘、限制 CPU 和内存。这些都是贴着操作系统的活。

用 JavaScript 干这些不是不行，但很别扭——需要不停调用外部命令、解析命令输出的文字。Go 在这方面是对口的：有成熟的库直接操作这些系统资源，而且编译出来是单个可执行文件，往机器上一放就能跑。

这个选择比较无争议。它属于「用对的工具干对的活」。

Rust：理由的一半没兑现

Rust 负责第 6 章讲的那个连线判断。当初选它的理由有两条：

1. 递归结构比较这类算法用 Rust 表达最清晰，而且编译产物小、能在浏览器后台线程里跑不阻塞画布
2. 同一份规则服务器也能用，避免前后端各写一套判断逻辑然后慢慢跑偏

第一条兑现了。第二条没有。经核实：服务器端没有任何地方调用这个判断，它只在网页端加载。

而且还有一层：Rust 那边其实写了一个更严格的连线判断，包含「方向必须是输出接输入」「可选的源不能接必填的目标」「双侧连接数上限」三项检查。但这个函数没有对外暴露的接口，浏览器根本调不到。实际生效的是网页端另写的一份 TypeScript 检查，它只查目标侧的连接上限，也没有「可选接必填」这一条。

这意味着什么

出现了两份规则：Rust 里那份严格的没在用，TypeScript 里那份宽松的在用。这正是当初想避免的「各写一套然后跑偏」，只是跑偏的形式变了——不是前后端跑偏，是同一个前端里两份实现跑偏。

诚实的结论是：按现状看，选 Rust 这件事偏重了。收益只剩表达力和产物体积，「一份规则两端共用」这个主要理由还没落地。

被问到时该怎么答

两个答案都不能用。别答「为了快」：Rust 和 JavaScript 之间传数据要来回转换格式，数据量小的时候这个开销可能比计算本身还大；而且这个项目没做过性能对照实验，答了会被追问数据。

也别答「为了不写两遍」——那确实是最初的目标，但上一节刚说了它没兑现。承认了「服务端没接入、形成了两份规则」，紧接着又说选它是「为了不写两遍」，等于自己打自己。

诚实的答法是把「目标」和「现状」分开说：

建议的表述

「当初想让前端和服务端共用一份判定规则，所以选了能编译到浏览器的 Rust。但服务端至今没接入，Rust 里那份更严格的判定也没暴露给浏览器，实际生效的是前端另写的宽松版。」

按现状如果重选，**纯 TypeScript 更简单**——除非先把 Rust 那份判定真正绑定到前端和服务端两侧，那时最初的理由才成立。」

第 五 部 分

走一遍完整流程

把前面所有零件按时间顺序串起来。这一章可以单独读——如果你只想快速理解整个系统，读它就够了。

第 18 章

点一次「运行」，系统里发生了什么

场景：你在画布上画好了那条周报工作流，点了「运行」。下面是从这一刻开始，系统内部发生的每一步。

阶段一：请求进门（几毫秒）

第 1 步，浏览器发出请求。网页把「运行这条工作流」这个意图打包成一个网络请求，带上你的身份凭据，发给服务器。

第 2 步，五道关卡。就是第 7 章那张表：初步解析、限流、验证身份、组织准入、角色权限。任一关不通过，请求就在这里被拒，你会看到一个明确的错误。

第 3 步，检查请求内容。用第 13 章讲的那份 Zod 规格检查请求体。不合规就返回 422，并指出哪个字段错了。

阶段二：登记与派单（几十毫秒）

第 4 步，开事务、设租户上下文。系统开启一个数据库事务，并在事务里告诉数据库「接下来的操作都属于这个组织」。这一步是数据隔离的关键，第 21 章详讲。

第 5 步，写执行记录。在数据库里创建一条「执行」记录，状态是「排队中」，并把当前工作流的快照一起存下来。

为什么要存快照

因为你可能在任务跑的过程中继续编辑画布。如果执行时去读「当前的画布」，就会出现跑到一半规则变了的荒唐情况。存快照等于把单子内容当场复印一份，之后你怎么改菜单都不影响这张已下的单。

第 6 步，提交事务，然后放队列。顺序不能颠倒——必须等数据库确认写入成功，才往队列里放单子。第 8 章解释过原因：否则工作进程可能取到单子却找不到记录。

第 7 步，立刻回复。服务器返回「收到了，执行编号是 XXX」。到这里请求就结束了，全程通常在一百毫秒以内。真正的活还没开始。

阶段三：网页开始盯着（并行发生）

第 8 步，网页建立实时连接。拿到执行编号后，网页通过 Socket.IO 订阅这个执行的进度。订阅时会带上「我已经收到第几号消息了」——首次是 0。

第 9 步，服务器回一份当前快照。因为从你点运行到订阅成功之间可能已经过了几十毫秒，这期间可能已经产生了事件。所以服务器先给一份「当前整体状态」，网页据此渲染初始画面。

阶段四：真正开始干活（几秒到几十分钟）

第 10 步，工作进程取单。某个空闲的工作进程从队列里取到这张单子，把执行状态改成「运行中」，并推一条事件出去。你的屏幕上此时开始有反应。

第 11 步，算出执行顺序。调度器读快照里的图，算出「哪些方块现在可以跑」——就是那些所有前置方块都已完成的。第一轮通常只有一个起点方块。

第 12 步，逐个执行方块。这是整个流程的主体，会循环很多轮。每个方块的执行大致是：

1. 收集输入——从上游方块的输出里取，按连线的对应关系装配好
2. 按方块类型干活——调 AI、查知识库、执行代码、发请求，各不相同
3. 记录结果——把输出存进数据库，状态改成「已完成」
4. 推事件——让你的屏幕上那个方块变成绿色
5. 算出下一批可以跑的方块，回到第 1 步

如果某个方块里是 AI 而且要执行命令，这一步会更复杂：系统要先起一个沙箱（第 10 章那台微型虚拟机），把 AI 的工具调用转发进去执行，再把结果拿回来。

第 13 步，遇到需要授权的操作就挂起。如果 AI 想写文件或改配置，执行会暂停，你的屏幕上弹出确认框。此时任务不是失败，只是在等。你批准了它继续，你拒绝了它按预设走，你一直不理它就按超时策略处理。

阶段五：中途可能出的岔子

如果某个方块失败了——比如调用的 AI 服务超时。系统会按配置重试若干次，间隔递增。仍然失败则把这个方块标记为失败，并根据配置决定是整条工作流终止，还是走「失败分支」继续。

如果你的网页断了——比如切到另一个网络。重连之后，网页会带上「我已经收到第 15 号消息了」重新订阅，服务器只补第 16 号之后的，不重放全部。所以刷新页面不会丢进度。

如果工作进程崩了——单子还在队列里，超时后被重新分配给别的进程。已完成的方块不会重跑，因为它们的状态和输出都已经落库了。

阶段六：收尾

第 14 步，加密敏感产物。AI 输出等敏感内容在写库之前加密。这里要诚实说明边界：加密发生在服务器进程里，所以服务器在那一刻是能看到明文的。它保护的是「数据库被拖走」，不保护「服务器被攻破」。

第 15 步，写审计日志。谁在什么时候运行了什么、结果如何，记进一张只能追加不能修改的表。

第 16 步，发通知。按你的偏好，通过站内消息、邮件或手机推送告诉你完成了。

第 17 步，回收沙箱。如果起过沙箱：会话型的用完直接销毁；持久型的只停机、保留磁盘，下次可以重启接着用，工作区内容会回写成快照。

整条链的一个共同特征

回头看这 17 步，有一个模式贯穿全程：每一步都先落库，再往下走。

这么做的成本是明显的——数据库写入次数多了很多，整体速度不如「全在内存里算完再存一次」。换来的是两个能力：

- 任何一步崩掉，都能说清崩在哪、当时的输入是什么
- 崩了之后能从断点继续，不用整条重跑

这是个典型的取舍：用性能换可观测性和可恢复性。对一个动辄跑几十分钟、还要花钱调用 AI 的系统，这个交换是划算的。如果是毫秒级的接口，就不划算了。

第 六 部 分

三处最费心的设计

前面讲的是「系统由什么组成」。这一部分讲三个具体问题。前两个（跨端契约、沙箱隔离）的共同特点是「错了不会当场报错，而是半年后变成事故」；第三个（多租户）的数据隔离同样闭环，但它的配额计费有一个[随时可被主动利用](#)的缺口——结论不同，见第 23 章。

第 19 章

怎么保证三个端说的是同一种话

系统有三个端：网页、手机、服务器。它们之间传数据要有约定。这一章讲这份约定怎么保证不走样。

问题的形状

服务器返回一条工作流的详情，里面有个字段叫 `description`（描述）。服务器的实现是「用户没填就返回空值」。网页端的开发者写代码时以为「这个字段一定有内容」，直接拿来显示。

结果：绝大多数工作流都填了描述，一切正常。某天有人建了一条没填描述的，那个页面白屏了。

这类问题为什么特别难防

三个特征让它成为最讨厌的一类 bug：一、不是立刻发生，可能上线三个月后才第一次触发。二、不是必然发生，取决于数据，测试环境往往碰不到。三、没有任何工具会警告你——服务器的代码是对的，网页的代码单独看也是对的，错的是两者之间那份没人维护的默契。

最直觉但错误的解法

写一份接口文档，让大家照着实现。这个做法失效得很快，原因不是人懒，而是文档和代码是两个东西，改了一个不会强迫你改另一个。三个月后文档就开始说谎，而且没人知道它从哪一句开始说谎。

这个项目的解法：让「定义」只有一份，其余全自动生成

核心思路是：不要让三个端各自维护一份理解，而是让其中一份成为唯一真相，其余从它自动产生。

具体链条是这样（这是那条 REST 接口链）：

1. 服务器用 Zod 写规格——这是唯一的真相来源，就是第 13 章讲的东西
2. 规格自动生成接口文档——不是人写的文档，是从规格导出来的机器可读文档
3. 文档自动生成类型定义——一个工具读文档，产出一份 TypeScript 类型文件
4. 网页端引用这份生成的类型——不许自己另写一份

结果是：服务器改一个字段，重新生成之后，网页端所有用到它的地方立刻编译报错。错误在几秒内暴露，而不是三个月后白屏。

打个比方

过去是设计部和生产部各存一份图纸，靠开会同步。现在是只有设计部那份是真的，生产部的图纸每天早上自动打印一次。设计改了尺寸，生产部当天就会发现自己的模具装不上，而不是等到装配线上才发现。

为什么说这比「写个测试检查一致性」更强

一个自然的替代方案是：写个测试，检查两边的定义是否一致。这个方案是有效的，但弱一档，因为测试可以被跳过——赶进度时加个跳过标记、或者干脆不跑，代码照样能上线。

而生成方案里，一致性是由编译保证的。网页端用了一个不存在的字段，编译就失败，打包产物根本产生不出来。编译不能被跳过，否则连东西都没有。

自动生成也有失灵的地方，必须显式处理

画布上的方块，其内部配置是「什么都可能有」的自由结构——不同类型的方块配置完全不同。还有一些坐标是「恰好两个数字」的固定组合。

这两种形状，接口文档的标准（OpenAPI 3.0）表达不了。生成出来的类型会退化成「一个啥都行的对象」，等于没有类型。

项目的处理方式是显式承认这一点：在网页端把这三个字段从生成的类型里**摘掉**，换成手写的画布专用类型，其余字段仍然用生成的。

```
type WorkflowDefinition = Omit<生成的类型, 'nodes' | 'edges' | 'viewport'>
& {
  nodes: CanvasNode[];      // 手写
  edges: CanvasEdge[];      // 手写
  viewport: Viewport | null; // 手写
};
```

Omit 的意思就是「从这个类型里去掉这几个字段」。这段代码的价值在于它是显式的——任何人看到都知道「这三个字段是特殊处理的，改的时候要注意」，而不是默默接受一个更宽松的类型然后忘掉。

还有另一条链：实时消息

上面讲的是 REST 接口。实时推送的消息走的是另一套机制，因为它不适合用接口文档那套流程。

做法是：把 Zod 规格和一批真实的示例数据（服务器实际发出的消息，存成 JSON 文件）放进同一个共享包。然后两侧各写一个测试：

- 契约包这侧：检查示例文件的集合、消息名列表、映射表三者两两完全对应——不许有示例没登记，也不许有登记没示例
- 服务器这侧：读同一批示例文件，跟自己真实发出的消息逐字段对比

效果是：服务器偷偷改了消息格式，两边的测试会同时变红。它没法只改一边而不被发现。

第 20 章

笼子是怎么焊住的

第 10 章讲了「为什么要用微型虚拟机」。这一章讲虚拟机之内的五道闸具体是怎么做的：读文件、写文件、解压缩包、网络、凭据。

一个重要的前提：这些检查在哪里执行

文件相关的检查不在服务器上，而在沙箱内部那个 Go 写的小程序里。这个区别很重要：如果检查在服务器上做，沙箱里的程序就有机会绕过它直接碰文件系统；放在沙箱内部、作为唯一的文件访问入口，才真正拦得住。

第一道闸：读路径只允许落在两个目录里

沙箱里只有两个目录是可写的：一个工作区目录，一个临时目录。判断一个路径是否在允许范围内，看起来只要检查开头就行，但有两个陷阱。

陷阱一：前缀伪装

如果只检查「路径是不是以 `/workspace` 开头」，那 `/workspace-evil/secret` 也会通过——它确实以那串字符开头，但它是完全不同的目录。

正确做法是连着路径分隔符一起比：要么完全等于 `/workspace`，要么以 `/workspace/` 开头。多这一个斜杠，就把伪装堵住了。

陷阱二：软链接

软链接 Symbolic Link

一种「快捷方式」式的文件。它本身只是一个指针，指向别处的真实文件。你读它，实际读到的是它指向的那个东西。

问题在于：一个位于允许目录里的软链接，可以指向目录外面。路径字符串看起来完全合法，实际访问到的却是外面的文件。

所以检查必须做两遍：

1. 先按字符串判断——拦住 `../../etc/passwd` 这类明显的往上跳
2. 再把软链接全部解开，得到真正的目标路径，重新判断一次

只做第一遍拦不住软链接；只做第二遍会漏掉一些情况。两遍都要。

第二道闸：写文件比读文件更严

写操作走一条独立的、更严格的检查，多了三件事：

- 不许把根目录本身当文件写——防止用一个文件覆盖掉整个工作区目录
- 如果目标已存在，它必须是个普通文件。不能是目录、不能是管道、不能是设备节点。往设备节点写东西可能产生意料之外的系统影响
- 如果目标不存在（要新建），就检查它的父目录——解开父目录的软链接，确认父目录确实是个目录、且在允许范围内

这里用到一个细节：判断「目标是否存在」时，用的是不跟随软链接的那种查询方式。因为如果跟随了，就无法区分「目标是个软链接」和「目标是它指向的那个文件」，而这个区别正是要检查的。

第三道闸：压缩包是独立的攻击面

解压一个压缩包时，包里每个条目都自带一个路径。如果直接按这个路径解压，一个精心构造的包能把文件写到任何地方去——包里写个 `../../etc/cron.d/evil` 就行。这是个有名的漏洞类型。

所以解压时对每个条目单独检查：拒绝绝对路径、拒绝含特殊字符的名字，拼接后重新规范化，确认结果仍在目标目录之下。而且压缩包里的「硬链接」条目也要单独检查，因为它也带一个路径。

第四道闸：网络只放行「用自己身份」发的包

虚拟机的网卡入口挂了一串规则，按优先级从上往下匹配：

```
优先级 10  普通网络包：源地址 和 源硬件地址都必须分配给它的 → 放行
优先级 11  地址询问包：硬件地址 和 声明的地址都必须对得上 → 放行
优先级 12  新版协议的包 → 全部丢弃
优先级 20  匹配一切 → 丢弃
```

前两条是白名单，第三条关掉一整类协议（因为用不到，关掉就少一个攻击面），最后一条是兜底——凡是没被前面明确放行的，一律丢弃。

为什么要同时校验两个地址？因为只查一个的话，沙箱可以伪装成别人：改一下自己声明的地址，就能冒充另一个沙箱发包。两个一起查，冒充就得同时改对两个，而另一个是硬件层面分配的。

另外还有一套防火墙规则以「整表一次性替换」的方式下发（避免中间出现规则不全的时间窗口），里面有两条关键的：沙箱之间互相访问，丢弃；访问内网地址段，丢弃。

第五道闸：凭据不对称

沙箱内部不持有能访问应用的证书。它需要回调时，必须经过一个受控的中转站。

中转站做的一件关键小事：转发之前先把请求里原有的身份信息剥掉，再换上沙箱自己的令牌。这样外部即使想借沙箱当跳板把凭据透传进去，也做不到——凭据在中转站就被扔了。

另外，服务器和沙箱管理器之间是双向验证：不只是客户端验服务端，服务端也要求并验证客户端的证书。这样任何一方被冒充都会被对方发现。

五道闸的共同原则

虽然五道闸的实现完全不同，但有一条原则是共同的：默认拒绝，明确放行。

这条原则的实际含义是：写规则时不去列举「不允许什么」，而是列举「只允许什么」，剩下全拒。两种写法的差别在于漏写一条的后果——前者漏写一条是一个漏洞，后者漏写一条只是少了一个功能。后者的失败方式安全得多。

第 21 章

怎么保证 A 公司看不到 B 公司的数据

这是多租户系统的核心问题。所有组织的数据都在同一个数据库的同一张表里，靠查询条件区分。

问题：一次遗漏就是一次数据泄露

正常的查询长这样：

```
SELECT * FROM agents WHERE organization_id = '当前组织' AND ...
```

如果某个人写某个查询时忘了加 `organization_id` 这个条件，这个查询就会返回所有组织的数据。代码能跑、测试可能也过（测试库里往往只有一个组织的数据），直到上线后有用户看到了别人的东西。

为什么「小心一点」不是解决方案

系统有 41 个业务模块、几百处数据库查询。要求每一处都不漏写一个条件，等于把安全押在「每个人每次都记得」上。这是纪律，而纪律必然会在某个时刻失效——新人不知道这条规矩、赶进度时忘了、复制粘贴时改漏了一处。

解法：两道锁，第二道假设第一道会失效

第一道：应用层建立可信身份

请求进来后，验证身份的那一会会把「这个请求属于哪个组织」写进请求对象里。关键是「验证之后」——在验证之前，请求里携带的任何组织信息都是不可信的，因为那是请求方自己填的。

然后系统开启一个数据库事务，在事务里下发两句话：

```
SET LOCAL ROLE authenticated;  
SELECT set_config('app.current_tenant', '组织ID', true);
```


第一句切换成一个受限的数据库角色。第二句把「当前组织」告诉数据库。最后那个 `true` 很关键：它表示这个设置只在本事务内有效，事务一结束自动失效。

如果那个 `true` 写成 `false` 会怎样

设置会留在数据库连接上。而数据库连接是复用的——这个请求用完还给连接池，下一个请求可能拿到同一个连接。于是 B 公司的请求可能带着 A 公司的组织上下文运行。这是个非常隐蔽且严重的漏洞，而区别只在一个参数。

第二道：数据库自己长了眼睛

行级安全 RLS

PostgreSQL 的一个功能：给表挂上规则，规定「什么身份能看到哪些行」。规则由数据库自己执行，在查询语句之外。

所以即使应用发来的查询完全没有组织条件，数据库也只会返回符合规则的行。规则的内容就是「这一行的组织字段，必须等于当前设置的组织」——正是上面第二句 SQL 设的那个值。

这个项目有 68 张表挂了这样的规则，而且是在正式的数据库迁移脚本里挂的（不只是在代码定义里写着），这一点经过核实。

打个比方

第一道锁是前台按包间号分发单据——靠流程保证不送错。第二道锁是每个包间的门自己认人——即使前台送错了，门也不会开。

第二道锁的价值恰恰在于它假设第一道会出错。如果前台永远不会错，第二道锁就是多余的；正因为前台是人（是代码，是纪律），第二道才有意义。

顺带讲一个巧妙的小设计：只能追加的表

审计日志有个要求：只能写，不能改、不能删。否则做了坏事的人可以把自己的记录删掉。

常见做法是加数据库触发器，在有人试图修改时报错。这个项目没用触发器，而是用了两层更简单的手段：

- 只给这张表建「查询」和「插入」两条规则，不建修改和删除的——没有规则允许的操作，数据库直接拒绝
- 数据库权限也只授予查询和插入

两层都不给，修改和删除就无从下手。比触发器简单，而且没有「触发器被误删」的风险。

怎么验证这套东西真的有效

光写规则不算完，得证明它生效。这个项目的做法是：测试时启动一个真实的数据库容器，逐条回放全部迁移脚本，把表和规则真实地建出来；然后分两种情形查询——带着组织上下文、和不带——断言跨组织查询返回空结果或者直接报权限错误。

用真数据库而不是模拟，是因为行级安全是数据库的功能，模拟出来的假数据库不会执行这些规则，测了也白测。

但这条线上有一个已知的缺口

上面讲的是数据隔离，这部分是完整的。但同一条链上还有个跟「配额」相关的问题，它不泄露数据，但能被用来攻击。这个问题放在第 23 章讲，因为它属于「还没修的东西」。

第七部分

面试时怎么说

这一部分不讲技术，讲的是怎么诚实且有力地描述自己的贡献。里面有几个反直觉的判断。

第 22 章

哪部分算你的贡献

你的处境是：系统设计是你做的，代码基本由 AI 生成。这一章讲怎么把这件事说清楚，以及为什么「说清楚」比「含糊过去」有力得多。

先说为什么不能宣称「代码是我手写的」

因为面试官验证归属只有一个手段：顺着因果往下追问。

他不会问「这是你写的吗」，他会问「这里为什么这么做」「不这么做会怎样」「边界在哪」。如果你宣称某个模块是自己写的，而对方随口问一句那个模块里某个具体机制的实现，你答不上来——损失不只是那一个模块，而是你之前所有陈述的可信度。包括那些你真的懂的部分。

这是个负期望的赌注

宣称范围越大，被问到自己不懂的部分的概率越高。而一旦被问穿一次，对方会重新审视你说过的每一句话。收益（听起来做得多）远小于风险（整体不可信）。

把代码按「你的介入方式」分三档

比「说了多少行」更有用的分法，是按失误的影响范围分。理由是：影响范围大的地方，必须由设计者亲自定边界，因为它的错误不会局限在一个模块内。

档位	包含什么
我主导设计与验收	跨端契约层、沙箱隔离边界。这两处的错误会跨语言、跨端、跨信任边界扩散
我定语义、按行为验收	多租户隔离、调度语义、事件协议、路由策略、类型引擎。设计意图是我定的，实现细节我按行为验收
生成为主、抽查为辅	41 个业务模块的增删改查、网页页面、手机端、插件市场与分账

第一档只有两处，这是核实之后的结果，不是凑不出三个。原本想把多租户放进第一档，但核实后发现它的配额部分有个没修的缺口（第 23 章讲），不满足「已验收」的标准，所以降档。

这个「宁可少一处，不凑数」的表述本身是有说服力的——它说明你对自己的判断有标准，而不是挑好听的说。

可以直接用的一段话

建议的表述

「系统设计是我做的，代码大量由 AI 生成。有两处边界是我主导设计并定验收标准的——跨端契约层、沙箱隔离边界，因为这两处的失误会跨端或跨信任边界扩散，不是单模块问题。

多租户的数据面是闭环的，68 张表挂了行级安全策略，事务上下文只读验签后的身份；但配额计费那一段排在认证之前、接受未验签的租户身份，这是我自己核实出来的缺口，还没修。

这几处的实现和测试我能讲清。具体某个业务模块的实现细节，我会答不上来。」

为什么这样说比宣称手写更强

三个原因：

1. 它是真的，所以经得起任意深度的追问
2. 它划定了范围，对方知道该往哪问，也知道哪里不必问——你不会被问到自己不懂的地方
3. 主动报出自己找到的缺口，是一个很强的能力信号。能说出「我核实后发现这里有个问题」，证明你真的核实过，而不是背了一遍文档

2026 年的面试官普遍接受 AI 辅助编程。真正稀缺的不是「手写了多少」，而是「知道哪里必须自己把关，以及为什么」。这个判断力很难伪造，而你恰好有。

这句话的兑现成本

说了「这几处我能讲清」，就必须真的讲得清。面试前把这两处边界、加上多租户那条链的实现和测试吃透一遍——就是本书第 19、20、21 三章的内容。这是可控的准备量。

而生成为主那一档，提前承认答不上来，比现场被问穿代价小得多。

已知还没修的问题

下面每一条都回源码核对过。声称某一块是自己的贡献之前，先确认它不在这张表上。

一个真实的安全缺口：配额可以被别人刷干

这一条最重要，单独讲。

问题在于两个检查的顺序。第 7 章那张表里，限流排在第 2 位，验证身份排在第 3 位——[限流在验身份之前](#)。

限流需要知道「这是哪个组织的请求」才能扣对应的额度。但此时身份还没验，它只能用请求里自己声称的组织身份——也就是未经验证的。

于是攻击链是这样：

1. 攻击者构造一个凭据，签名部分是随便填的垃圾，但里面写着受害者的组织 ID
2. 用它反复请求任意一个受限流的接口
3. 限流那一关读到受害者的组织 ID，把受害者今天的额度扣掉一次
4. 然后才走到验身份那一关，签名不对，返回 401 拒绝
5. 但额度已经扣了。重复几千次，受害者当天的额度就被刷干了

这个漏洞的性质

它不泄露任何数据——数据隔离那两道锁是完好的。它是一个[可用性攻击](#)：让受害者的正常请求被系统自己拒绝。

严重性在于两点：不需要任何有效凭据（谁都能干），以及可以精确指定受害者（不是随机破坏）。

修的方向有两个，选一个即可：把配额计费移到验证身份之后；或者对未验证的请求只按来源 IP 计数，等验证通过后再按组织计费。修完需要补一个回归测试：用签名无效的凭据打接口，断言受害者组织的额度计数不变。

其余已核实的缺口

缺口	现状
类型引擎没接到服务器	第 17 章讲过：服务器完全没调用它；Rust 里那份更严格的连线判断没有对外接口，浏览器调不到，实际生效的是网页端另写的宽松版
对话的断线补播没接通	工作流那边是好的——重连时带上「已收到第几号」，服务器只补差量。但对话那边只声明了这个字段，订阅时不上报、收到消息也不更新，等于这个能力没接上
一个流式适配器没在用	代码和单元测试都有，但整个服务器没有任何地方调用它
插件命令行工具的「发布」不上传	它只做签名并把包写回本地，不上传到任何地方；但命令的说明文字写的是「发布到市场」
文档说错了哪张表只能追加	文档称租户密钥表是只能追加的，实际不是——迁移脚本里给了完整的修改和删除规则。真正只能追加的是两张审计表
审计写入口不唯一	治理模块另有一条直接写审计表的路径。这是有意的——主事务会因为拦截而回滚，审计必须在独立事务里落地，否则「拦了但没留痕」。失真的是文档措辞「唯一正式写入口」
优化建议不能采纳	四类建议写入的位置对执行不产生实际影响，所以前后端用同一份空名单显式禁用了「采纳」按钮，只留「忽略」。这是刻意的，不是残缺

两条方法纪律

这份清单在整理过程中出了两次错，教训比清单本身更有价值。

一、别拿自己的旧笔记当现状

清单里原本有一条「某个面板没有被使用」，是从三周前的一份工作记录里抄来的。回源码一看，那个面板早就被正式引入并渲染了。

二、别拿测试替身当生产实现

讲沙箱文件边界时，最初引用的是一份只在特定环境变量打开时才会启用的测试用实现。真正的生产实现在沙箱内部那个 Go 程序里，而且严格得多——多了「不许把根目录当文件写」「已存在的目标必须是普通文件」「压缩包条目单独校验」这些检查。

这两条教训的共同点

都是把「代码存在」当成了「代码生效」。确认一段代码是否在生产路径上，比读懂这段代码更重要。

判断方法很具体：它有没有被调用？调用它的地方有没有被环境变量门控？它在 `testing/` 或 `__tests_` 目录下吗？这三个问题问完再下结论。

面试前逐条回源码核对一遍，成本几分钟，收益是不在现场说出一句已经不成立的话。

全书完。如果你想再往深处看，同目录下还有两份给内行看的册子：《系统设计与开放问题》讲设计层面的取舍与未解决的问题，《实现细节参考册》按模块列出数据表、接口和协议细节。它们假设读者已经具备本书第二部分的名词基础。